Inside the transformer

The architecture behind every chatbot you have used

Architecture · A worked example · Dialogue · Code

Where we left off

Two problems, one solved.

1

The bottleneck — solved

One context vector could not carry a long sentence. Attention fixed it by keeping the encoder state after every word and letting the decoder choose.

2

The sequencing — still there

Both networks were still LSTMs. Word four could not start until word three finished, so training time grew with length and hardware could not help.

3

The question

If attention is doing the work of relating words to each other, what is the recurrence still for?

Nothing. Delete it.

Keep attention, throw the recurrence away, and run every position at the same time. That is the transformer — 2017, "Attention is all you need".

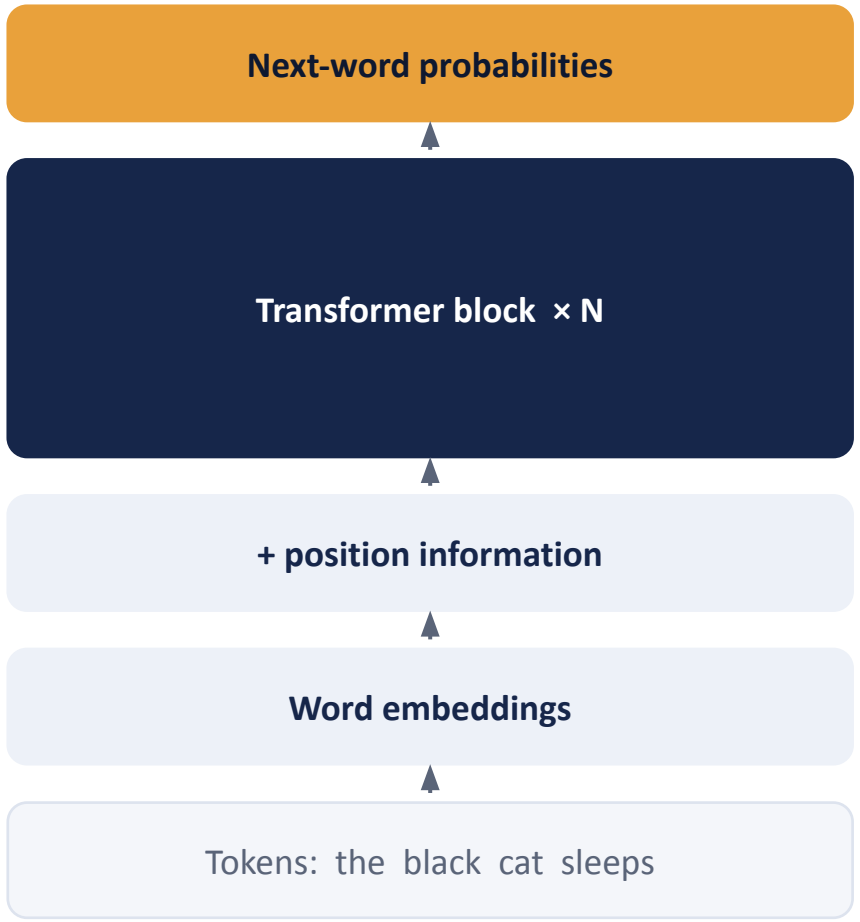
PART ONE

The architecture

Six pieces. Each one exists to fix a problem the previous piece created.

The shape of the whole thing

Words go in at the bottom. A probability for the next word comes out of the top.



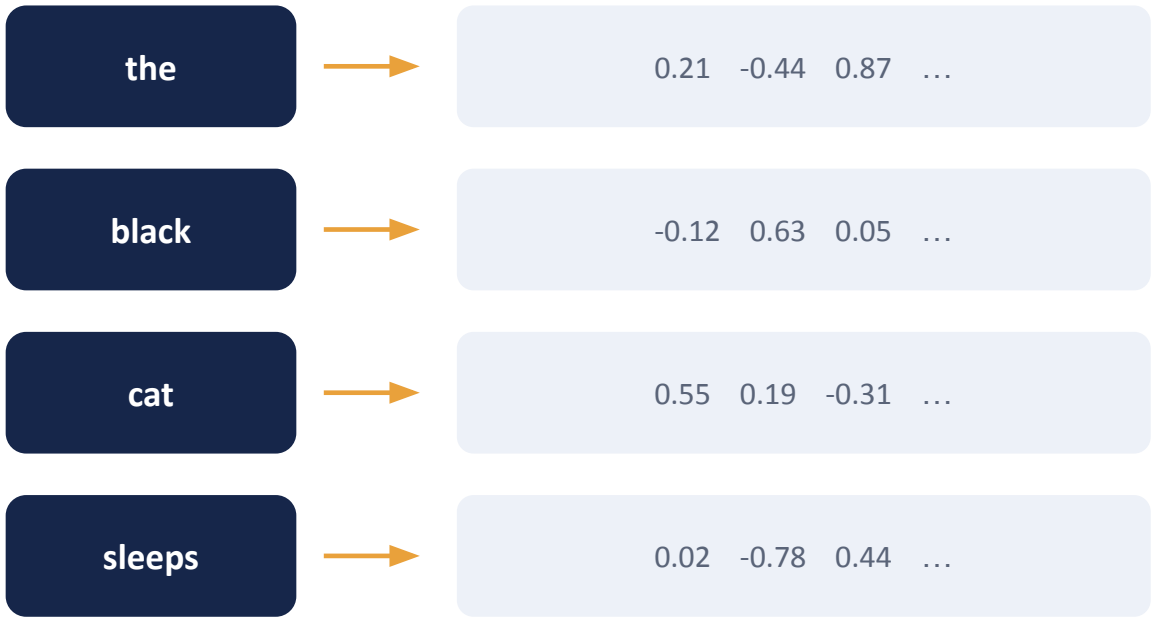
inside one block:

- Multi-head self-attention
- Add & normalise
- Feed-forward network
- Add & normalise

GPT-3 stacked this 96 times.

Step 1 — words become vectors

A lookup table, learned during training.



Every word gets a row

Typically 768 numbers, or 4096 in a large model. Nothing is hand-designed — these start random and are learned like any other weight.

Similar words end up close together

Not because anyone arranged them that way, but because words used in similar contexts get similar gradients.

Step 2 — telling it the word order

Attention is a weighted average, and averages do not care about order.

Without a position signal these two are the same input:

"Zuko made his uncle tea"
"his uncle made Zuko tea"

Same bag of words. Very different afternoon.

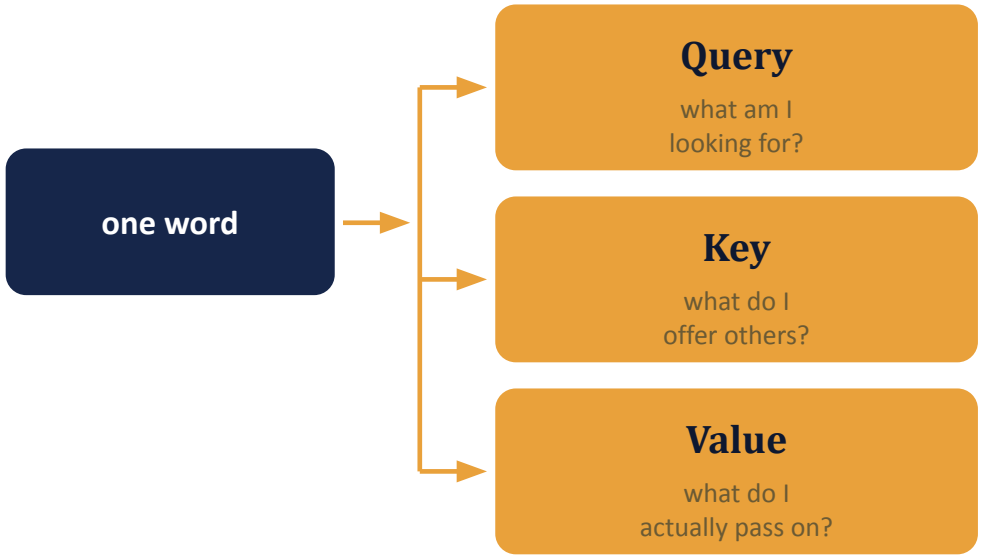
The fix is almost crude: add a position vector to each word embedding before the first block.

The original paper used fixed sine and cosine waves of different frequencies, so position 5 has a distinctive pattern that also tells you something about being near position 6.

Most models today simply learn a vector per position, or encode relative distance inside attention itself (RoPE).

Step 3 — self-attention, and the one new idea

Each word attends to the other words in its own sentence.



three different learned matrices, applied to the same vector

You already know two of these.

In the attention we built last class, the query was the decoder state and the key and the value were the same encoder vector.

What is new: separating key from value.

A word can now be findable by one property and contribute a different one. "cat" can advertise "I am a noun, singular, masculine" so adjectives find it, while passing on what it means.

Self-attention just means the queries and the keys come from the same sentence.

The same three steps as last class

Score, normalise, blend — now with a dot product.

1

Score

Multiply each query by every key: score = q · k. High when the two vectors point the same way. Then divide by the square root of the dimension, or the numbers get large, softmax saturates, and gradients die.

2

Normalise

Softmax across each row, so the weights are positive and sum to 1. Same picture as the heat map from last class — but now every word has a row, not just the output words.

3

Blend

Weighted average of the value vectors. That blend replaces the word's own vector on the way out, so after one block every word carries information about its neighbours.

Step 4 — do it several times at once

One attention pattern is not enough. Run eight, in parallel.

head 1	head 2	head 3	head 4
adjectives → nouns	*verbs → subjects*	*the previous word*	*something we cannot name*

concatenate, then one more matrix to mix them back together

Each head gets its own Q, K and V matrices and a slice of the dimensions — 8 heads of 64 rather than 1 head of 512. Same cost, several relationships at once. The labels above are illustrative: heads are not assigned roles, and many resist any clean description at all.

Step 5 — hiding the future

The single difference between an encoder and a decoder.

Set the greyed-out scores to minus infinity before the softmax, and they come out as exactly zero.

Why: the model is trained to predict the next word. If "cat" could see "sleeps", the answer would be in the input and it would learn nothing.

Remove the mask and you have BERT. Keep it and you have GPT. Everything else about the block is identical.

Step 6 — the parts nobody puts on the poster

Without these three, a deep stack simply will not train.

1

Feed-forward

After attention, each position goes through a small two-layer network on its own.

Attention mixes information between words. This part thinks about it.

It is also where most of the parameters live.

2

Residuals

Every sub-layer adds its output to its input rather than replacing it.

So the original signal always has a clean path to the top, and gradients have a clean path back down.

3

Layer norm

Re-centre and re-scale the vectors at each step.

Keeps the numbers in a sane range across dozens of layers. Unglamorous, and non-negotiable.

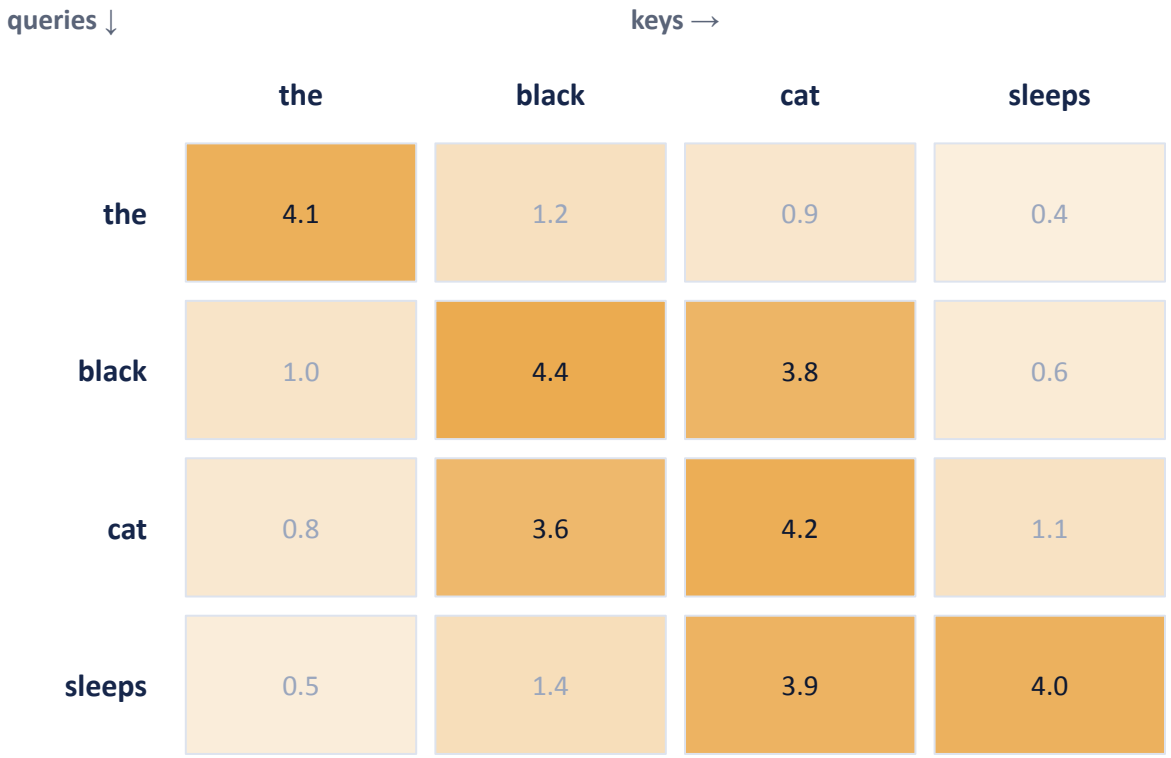
PART TWO

A worked example

One head, four words, real arithmetic.

Every word scores every word

q · k for all 16 pairs — this is one matrix multiply.



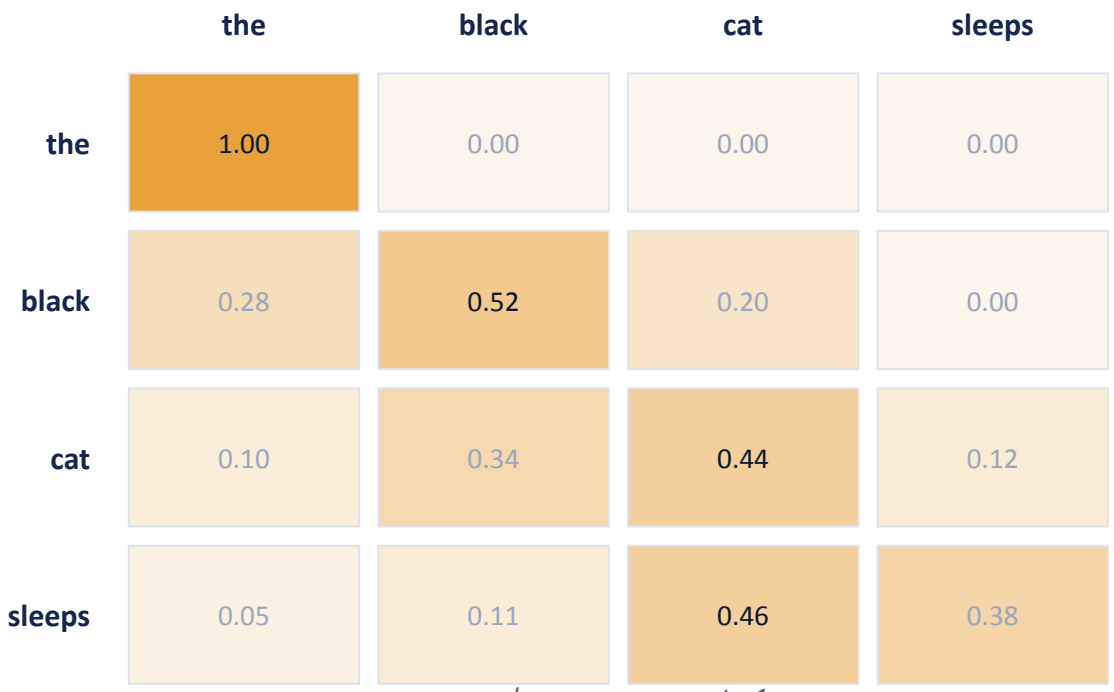
Read row 2: the query from "black" against every key.

It scores 4.4 against itself and 3.8 against "cat" — the noun it modifies — and almost nothing against "sleeps".

These are raw dot products. They are not yet weights: they can be any size, and they do not sum to anything.

Scale, mask, softmax

Now they are weights.



each row now sums to 1

Row 2 in words.

While processing "black", the model builds a new vector from 52% of black's own value, 28% of "the", and 20% of "cat" — and nothing from "sleeps", which is masked out.

That blend leaves this block as the new representation of "black". It is no longer just the word. It is the word in this sentence.

One block gives context.
Sixty give understanding.

Block 1 lets "black" see "cat". Block 2 sees a sentence where every word already knows its neighbours. By block 40 the vectors encode things nobody has a good name for.

PART THREE

From architecture to dialogue

Nothing new gets added. Something gets formatted.